MAYDAY CYBERSECURITY

Security Agentic Systems Brief

Detection ▪ Triage ▪ Deception

Systems: prompt-shield ▪ intelligent-defender ▪ honeypot-orchestration

Classification: Internal — Strategic & Technical

Date: March 6, 2026

Prepared by **Mayday Cybersecurity**

Table of Contents

1. Executive Summary
2. Agentic Design Patterns & Rationale
3. System Architecture
4. Integration Flow
5. Prompt Injection Defense
6. External Input Scanning Architecture
7. Key Differentiators

1. Executive Summary

Mayday operates three LangGraph-based security agents that form a closed-loop **detection, triage, and deception** pipeline:

prompt-shield	intelligent-defender	honeypot-orchestration
5-LAYER DEFENSE-IN-DEPTH	SOC AUTOMATION (1:300)	MULTI-CLOUD DECEPTION

prompt-shield — Defense-in-depth middleware that scans all input to and output from AI agents for prompt injection attacks, PII leakage, and behavioral anomalies. Operates as a sidecar/middleware layer wrapping every deployed agent.

intelligent-defender — SOC coordinator that ingests security events from CloudWatch, GuardDuty, Splunk, and SIEM feeds; correlates against threat intelligence; triages alerts with severity scoring; executes automated containment; and dispatches deception tasks.

honeypot-orchestration — Multi-cloud deception agent that analyzes architecture via AppAcuity risk diagrams and deploys honeypots across AWS, O365, and Google Workspace. Monitors interactions and generates actionable intelligence.

> **Closed-Loop Architecture**
>
> prompt-shield **detects** threats at the agent boundary, intelligent-defender **triages and responds**, and honeypot-orchestration **deploys deception infrastructure** to gather intelligence on attacker behavior. Canary token fires feed directly into intelligent-defender as CRITICAL alerts, which can trigger automated honeypot deployment near the affected resource.

All three systems are built on LangGraph `StateGraph` with typed state, deterministic node execution, and checkpointing — chosen specifically for the auditability and reproducibility requirements of security operations.

2. Agentic Design Patterns & Rationale

2.1 StateGraph — Deterministic Node Execution

All three agents use `langgraph.graph.StateGraph` as their orchestration primitive, chosen over autonomous agent loops (e.g., ReAct with unconstrained tool use) for security-critical reasons:

- **Deterministic execution order.** Each node fires in a defined sequence. In intelligent-defender: `log_ingestion → threat_correlator → alert_triager → responder → reporter → deception_coordinator`.
- **Typed state.** Every piece of data flowing through the graph is defined as a `TypedDict` with explicit fields and types, making it auditable and testable.
- **Checkpointing.** LangGraph's built-in checkpointing records every state transition, enabling forensic reconstruction of any security decision.

2.2 Parallel Fan-Out / Fan-In

- **prompt-shield IngressGate** runs three scanners in parallel (`MLClassifier`, `RegexScanner`, `PerplexityChecker`), then merges scores into a single `ScanResult`. A failing ML classifier does not block the regex scanner from catching a known signature.
- **honeypot-orchestration** fans out deployment across three platforms (AWS, O365, GWS) via conditional routing, then converges at the `monitor` node.

2.3 Conditional Routing

honeypot-orchestration uses conditional edges to skip platforms with no pending deployments. intelligent-defender uses severity-based routing: events scoring below the threshold (default 30) are dropped entirely.

2.4 Typed State Accumulation

Each system defines a rich `TypedDict` state that grows through nodes:

SYSTEM	STATE FLOW
intelligent-defender	SecurityEvent[] → ThreatIndicator[] → Alert[] → ResponseAction[] → DeceptionTask[] → IncidentReport[]
honeypot-orchestration	AppAcuityDiagram → PlacementRecommendation[] → HoneypotDeployment[] → ThreatEvent[] → ActionableIntelligence[]
prompt-shield	Messages → {ML, Regex, Perplexity} → AggregatedScores → PolicyDecision (ALLOW|WARN|BLOCK|ESCALATE)

2.5 Defense-in-Depth Layering

prompt-shield implements 5 sequential defense layers, each independent:

LAYER	NAME	FUNCTION	SCANNER
1	**Ingress Gate**	Pre-LLM input scanning	MLClassifier, RegexScanner, PerplexityChecker
2	**System Hardening**	Constrain agent behavior	NeMo Guardrails Colang rules (planned)
3	**Canary Tokens**	Zero false-positive breach detection	CanaryManager + CanaryScanner via canari-llm
4	**Egress Gate**	Post-LLM output + tool call scanning	PIIScanner, CanaryScanner, ToolWhitelistValidator
5	**Privilege Enforcement**	Least-privilege per tool	SecurityProfile tool whitelists + rate limits

Design Principle

"Perfect prevention is impossible" (OWASP). Optimize for fast detection and response. No single layer is sufficient.

2.6 Closed-Loop Feedback

prompt-shield DETECTS → intelligent-defender TRIAGES → honeypot-orchestration DEPLOYS → ThreatEvent FEEDS BACK

2.7 Human-in-the-Loop Gates

- **intelligent-defender:** Containment actions (block IP, isolate instance, revoke credentials) are recorded with status `"pending"` for human approval by default.
- **honeypot-orchestration:** `SecurityGuardrails` requires human approval for `MODIFY_INFRA`, `DESTROY`, or `CREDENTIAL_OP` actions.
- **prompt-shield:** `ESCALATE` action routes alerts to human SOC analysts via Slack and PagerDuty.

2.8 Security-as-Code Profiles

Per-agent YAML security profiles loaded by `ProfileLoader` specify scanner thresholds, tool whitelists, canary token configuration, default actions, escalation channels, and rate limits. These are version-controlled alongside application code and reviewed in PRs.

3. System Architecture

3.1 prompt-shield: 5-Layer Defense-in-Depth

COMPONENT	STATUS	DESCRIPTION
RegexScanner	Implemented	8 patterns: ignore_instructions, system_override, DAN jailbreak, role_hijack, instruction_injection, base64_payload, data_exfil_request, encoding_evasion
SecurityProfile / ProfileLoader	Implemented	YAML loading, per-agent profile configuration
AlertPayload	Implemented	Pydantic model for SOC webhook payloads
MLClassifier	Phase 1	Guardrails AI PromptInjection validator wrapper
PerplexityChecker	Phase 2	Adversarial suffix detection via token distribution
PIIScanner	Phase 2	8 PII/secret types via Guardrails AI
IngressGate / EgressGate	Scaffolded	Parallel scanner coordination
CanaryManager	Scaffolded	Token generation, injection, rotation via canari-llm
PolicyEngine	Scaffolded	Score aggregation against SecurityProfile thresholds

3.2 honeypot-orchestration: Multi-Cloud Deception

placement_analyzer → aws_deployer → o365_deployer → gws_deployer →

monitor → alert_respond

Security Guardrails (fully implemented): Input validation (credential detection, platform allowlisting, per-run deployment limits), action authorization (5 severity levels with human approval gates), rate limiting, output sanitization, tamper-evident SHA-256 audit entries, and credential isolation via AWS Secrets Manager.

3.3 intelligent-defender: SOC Automation

Scale target: 1 operator managing 300 nodes

log_ingestion → threat_correlator → alert_triager → responder → reporter → deception_coordinator

Alert scoring (implemented): Base scores: critical=90, high=70, medium=40, low=15. +20 boost if source IP matches known-malicious from threat indicators. Events below threshold (default 30) are dropped. Score ≥80 = critical, ≥60 = high, ≥40 = medium, else low.

4. Integration Flow

4.1 Detection-to-Deception Pipeline

End-to-End Flow

1. prompt-shield detects injection (canary fire, regex match, ML classification)
2. AlertPayload dispatched to intelligent-defender webhook
3. intelligent-defender ingests, correlates CTI, scores alert, executes containment
4. deception_coordinator dispatches to honeypot-orchestration
5. Honeypots deployed near affected resources
6. ThreatEvent from honeypot interaction feeds back to intelligent-defender
7. prompt-shield updates scanning profiles based on new threat intel

4.2 Canary Token Fire Scenario

1. prompt-shield **EgressGate** detects canary token in LLM output via `CanaryScanner`
2. `PolicyEngine` evaluates: canary fire = automatic `ESCALATE` (confidence: 1.0, zero false positives)
3. `AlertPayload` generated with `severity: "CRITICAL"`, `detection_layer: "canary"`
4. Alert dispatched to: intelligent-defender webhook, Slack #security-alerts, PagerDuty
5. intelligent-defender scores event at 90+ (critical base + CTI correlation)
6. Responder executes containment: blocks session/IP, triggers credential rotation
7. deception_coordinator dispatches honeypot deployment near affected resource
8. Honeypots gather intelligence on attacker's lateral movement

4.3 Alert Payload Contract

```
class AlertPayload(BaseModel):
    event_type: str = "prompt_injection_detected"
    timestamp: datetime
    severity: str           # LOW, MEDIUM, HIGH, CRITICAL
    agent_id: str
    detection_layer: str    # ingress, egress, canary
    detection_details: DetectionDetails
    context: AlertContext
    recommended_action: str
```

5. Prompt Injection Defense

5.1 Threat Landscape

Prompt injection is the **#1 threat** to LLM applications per OWASP LLM01:2025, appearing in over 73% of LLM security assessments. OWASP states: "it is unclear if there are fool-proof methods of prevention." This makes defense-in-depth the only viable strategy.

5.2 Threat Categories

ID	THREAT	SEVERITY	MITRE ATLAS
T1	Direct Prompt Injection — override system instructions	CRITICAL	AML.T0051.000
T2	Indirect Prompt Injection — malicious instructions in external data	CRITICAL	AML.T0051.001
T3	AI-Induced Lateral Movement (AILM) — injection propagates via shared data	CRITICAL	—
T4	Tool Abuse via Injection — coerce agent into dangerous tool calls	CRITICAL	—
T5	Multi-Agent Propagation — injection in one agent's output infects another	HIGH	—
T6	Multimodal Injection — hidden instructions in images, PDFs	MEDIUM	—

5.3 Implementation Roadmap

PHASE	TIMELINE	GOAL
Phase 1: Foundation	Weeks 1-3	Basic input scanning + canary tokens. LangGraph scaffolding, Guardrails AI, canari-llm, regex scanner, YAML profiles.
Phase 2: Output Monitoring	Weeks 4-6	Post-LLM scanning + CI/CD gates. Tool whitelist validator, PII scanner, Garak in CI, NeMo Guardrails.
Phase 3: Observability	Weeks 7-9	Full SOC dashboard + automated response. Prometheus, Grafana, OpenTelemetry, alert routing, PagerDuty.
Phase 4: Advanced	Weeks 10-12	Custom ML + red-team validation. Custom classifier, AILM correlation, secondary guard model.
Phase 5: Continuous	Ongoing	Monthly Garak red-team, quarterly ML retraining, 30-day canary rotation, OWASP/MITRE tracking.

Current status: Phase 1 in progress. RegexScanner (8 patterns), SecurityProfile/ProfileLoader (YAML), and AlertPayload (Pydantic) are implemented. Remaining components scaffolded with typed interfaces.

6. External Input Scanning Architecture

6.1 IngressGate as Universal Middleware

Every external input — webhooks, email ingestion, CRM triggers, customer chat — passes through the `IngressGate` before any data reaches an LLM. The IngressGate coordinates parallel scanner execution and returns a single `ScanResult` with an `Action` decision.

6.2 Integration Patterns

LangGraph Native Middleware

```
agent = create_react_agent(
    model=model,
    tools=tools,
    middleware=[
        prompt_shield_ingress_middleware,
        prompt_shield_egress_middleware,
        canary_token_middleware,
    ],
)
```

Decorator Pattern

```
@shield_middleware
async def my_node(state, config):
    ...  # pre-LLM scan + post-LLM scan wrapped automatically
```

FastAPI / Lambda Middleware

```
shield = ShieldPipeline.from_config("shield-config.yaml")
result = await shield.scan_ingress(messages=..., agent_id=...)
if result.action == "BLOCK":
    return shield.blocked_response(result)
```

6.3 Scanner Inventory

SCANNER	TYPE	STATUS
RegexScanner	8 compiled patterns (CRITICAL to MEDIUM severity)	Implemented
MLClassifier	DeBERTa/DistilBERT fine-tuned (90-95% detection, 5-50ms)	Phase 1
PerplexityChecker	Token distribution anomaly detection for adversarial suffixes	Phase 2
PIIScanner	8 PII/secret types (email, phone, SSN, API keys, etc.)	Phase 2
CanaryScanner	Zero false-positive exact match (canari-llm, 30-day rotation)	Phase 1
ToolWhitelistValidator	Per-agent tool name + parameter constraint validation	Phase 2

6.4 Canary Token Architecture

1. **Generation:** canari-llm creates deterministic fake secrets (API keys, AWS keys, emails) that look authentic
2. **Injection:** Embedded into agent system prompts — the LLM sees them as legitimate secrets
3. **Scanning:** EgressGate checks every LLM output for exact token matches — any match is a **zero false-positive** signal
4. **Alerting:** Canary fires trigger CRITICAL alerts via webhook + PagerDuty
5. **Rotation:** Tokens rotate every 30 days (configurable per profile)
6. **Forensics:** All activity stored in local SQLite and forwarded to DynamoDB

6.5 Policy Engine

- **Any canary fire:** Automatic `ESCALATE` (confidence 1.0)
- **CRITICAL regex match:** Automatic `BLOCK`
- **ML above threshold:** `BLOCK` or `ESCALATE` per profile
- **Multiple MEDIUM detections:** Weighted aggregation can trigger `WARN` or `BLOCK`
- **Below all thresholds:** `ALLOW`

7. Key Differentiators

Zero False-Positive Detection

CANARY TOKENS PROVIDE MATHEMATICALLY ZERO FALSE-POSITIVE BREACH DETECTION

Guardrails from Day 1

AI SECURITY BAKED IN BEFORE ANY DEPLOYMENT NODE IS WRITTEN

Security-as-Code

YAML PROFILES, TOOL WHITELISTS, AND COLANG RULES IN VERSION CONTROL

Deterministic Auditability

STATEGRAPH + CHECKPOINTING = FULL FORENSIC RECONSTRUCTION

Founding Design Principle

"Automation without security is dangerous. Security without automation is exhausting." — Every agent gets shield coverage. No exceptions. Every security decision is logged, traced, and metriced. Changes to security posture go through the same code review process as application changes.

Closed-Loop Intelligence

Unlike standalone detection or response tools, the three-agent pipeline creates a self-reinforcing intelligence cycle. Attacks detected by prompt-shield generate threat indicators that improve intelligent-defender's CTI correlation, which drives honeypot placement that captures attacker TTPs, which feeds back into prompt-shield's scanning profiles. Each iteration sharpens the defensive posture.

Design Principles

1. **"Automation without security is dangerous."** Every agent gets shield coverage.
2. **Zero-trust agents.** Treat every LLM as a potentially compromised endpoint.
3. **Defense in depth.** Stack ML classifiers, canary tokens, tool validation, and behavioral monitoring.
4. **Least privilege always.** Tool call whitelists enforced by shield, not by the agent itself.
5. **Detection over prevention.** Perfect prevention is impossible. Optimize for fast detection.
6. **Observable by default.** Every shield decision is logged, traced, and metriced.
7. **Security as code.** Profiles, whitelists, and rails are version-controlled and PR-reviewed.

Mayday Cybersecurity — Help is on the way.
Confidential — Internal Use Only — March 2026